

Slash Commands Globales

Guide complet — 19 commandes Claude Code utilisables sur tous tes workspaces

19

COMMANDES

6

CATÉGORIES

10+

AGENTS MAX
PARALLÈLES

∞

PROJETS
COMPATIBLES

Emplacement : `~/ .claude/commands/` · disponibles sur tous tes projets

Override : si `.claude/commands/<nom>.md` existe dans le projet, il prend le dessus

Généré : 2026-05-15 · Hearst OS workspace

Sommaire

● Bootstrap & déploiement

/setup	Bootstrap projet complet (Supabase + Railway + Vercel)
/electron	Wrapper Electron desktop avec sélecteur env au boot
/ready	Preflight go-live (12 agents, services, crédits, deploy)

● Audit & qualité

/audit	Audit transversal sec/perf/arch/deps
/qa	Audit UI/UX (alignements, spacing, états, responsive)
/flow	Audit flows utilisateur (navigation, friction, CTA)
/pixel-audit	Audit DS pixel-perfect (10 agents)
/ui	Audit visuel d'un seul fichier
/nettoyage	Dead code, magic numbers, dettes lint
/test	Couverture + génération specs prioritaires

● Design

/design	Mode design + sandbox HTML + prompts Sonnet/Gemini
/heist	Scrape DS d'un site → theme drop-in
/skin	Heist + intégration admin (UI switcher par les users)

● Compréhension

/brief	Briefing 30s (stack, branche, WIP, next)
/map	Cartographie routes, stores, deps, API
/syscheck	Inspection système macOS (zombies, ports, caches)

● Orchestration & maintenance

/trio	Orchestrateur Opus → 3 Sonnet manuels parallèles
/tri	Trie les HTML/MD orphelins (auto-archive)

● Stratégie

/horizon	Stack vs état de l'art 2026 (10 agents)
----------	---

Bootstrap & déploiement

/setup

<nom-du-projet>

Bootstrap d'un nouveau projet de zéro à prod en une commande. Aucune question posée.

CE QUE ÇA FAIT

- Détecte le stack (Next.js, SvelteKit, etc.) et les ports libres
- Crée le projet Supabase (tables + RLS + storage)
- Crée le projet Railway (DB + Redis)
- Configure Vercel (link + env vars production + preview)
- Génère `.env.local` avec toutes les clés du catalogue global
- Génère `CLAUDE.md` adapté au stack
- Premier commit + deploy Vercel

QUAND L'UTILISER

Sur un nouveau projet vide ou existant qui n'a jamais été configuré.

CATALOGUE GLOBAL

`~/.claude/api-config/SERVICES.md` — toutes tes clés API réutilisées

SORTIE

Liens Supabase, Railway, Vercel + chemin du `.env.local`

/electron

[build|test|publish]

Wrappe ton app web en application desktop Electron signée, avec sélecteur d'environnement au boot.

CE QUE ÇA FAIT

- Génère `electron/main.ts`, `preload.cts`, `splash.html`
- Splash au boot avec 2 boutons : **Local** (localhost:port) / **Production** (URL Vercel)
- Auto-update via GitHub Releases (electron-updater)
- Notarize macOS, signature, DMG + ZIP + NSIS + ApplImage
- Tests Playwright sur le flow boot → app
- Menu "Changer d'environnement..." pour revenir au splash

ARGUMENTS

`vide` = setup complet · `build` = release signée · `test` = Playwright ·
`publish` = release GitHub + auto-update

DÉTECTION AUTO

Port (depuis `.env.local`), URL prod (`vercel inspect`), nom (`package.json`)

/ready

[redeploy]

Preflight check complet avant go-live : ping tous services, vérifie crédits, redéploie si décalé.

12 AGENTS EN
PARALLÈLE

- Deploy Vercel (HEAD local vs prod, build status)
- Build Next.js + Electron
- Backend (Supabase, Railway, Redis)
- LLM providers (Anthropic, OpenAI, DeepSeek) — rate limits
- Tools (Composio, Exa, Tavily, Browserbase)
- Media (ElevenLabs, Runway, Deepgram, Recall, Hume, fal)
- **Crédits & quotas** — alerte si solde < 20%
- Inngest functions registered + crons actifs
- Auth & sessions (NextAuth, OAuth)
- Observabilité (Sentry, Langfuse, Axiom reçoivent ?)
- Smoke E2E (login → chat → LLM)

RAPPORT HTML

Bandeau **READY / NOT READY** + 12 tuiles + crédits faibles avec liens dashboards + versions désynchro

MODE REDEPLOY

Avec `/ready redeploy` : lance `vercel --prod` auto si HEAD désaligné

Audit & qualité

/audit

[security|perf|arch|deps]

Audit transversal multi-axe. 4 agents en parallèle. Rapport HTML avec P0/P1/P2.

4 AGENTS

Sécurité (OWASP, secrets, auth) · **Performance** (fichiers > 300L, , barrel imports) · **Architecture** (circulaires, couplage, typage) · **Dépendances** (npm audit, depcheck)

SORTIE

Findings classés P0/P1/P2 avec quick wins + plan de remédiation en 3 batches

/qa

[chemin|module]

Audit QA UI/UX ultra-détaillé. 3 agents. Score /10. Senior QA perfectionist.

3 AGENTS

Layout (alignement, spacing, magic numbers) · **États** (loading, error, empty, focus, hover) · **Visual** (couleurs, radius, shadows, cohérence)

SCORE

Base 10 – 2×P0 – 0.5×P1 – 0.1×P2 (plancher 0)

/flow [flow-id]

QA flows utilisateur (onboarding, auth, upload, settings...). 3 agents. Score UX /10.

3 AGENTS

Navigation (dead-ends, retour, transitions) · **Product logic** (clics, redondances, destructions sans confirm) · **Feedback** (loaders, erreurs, modales, focus trap)

/pixel-audit (sans arg)

Audit DS pixel-perfect transversal. 10 sous-agents en parallèle. La grosse artillerie.

10 AGENTS

Tokens · Typo · Spacing · Couleurs · Radius/shadows · Motion · Voix · Primitives · États/a11y · Stabilité/cross-refs

SORTIE

Top 10 quick wins + plan 3 batches (P0 / P1 / P2)

/ui <chemin/fichier.tsx>

Audit visuel d'un **seul fichier**. Read-only. Tableau magic numbers ligne par ligne.

SORTIE

Tableau exhaustif `ligne | code | token correct | note` + 3 actions de correction ordonnées par impact

/nettoyage [chemin|glob]

Nettoie le codebase. 5 agents. Dead code, magic numbers, dettes lint, TODO obsolètes.

5 AGENTS

Exports orphelins (knip) · Dead code TS (ts-prune) · Magic numbers · Dettes ESLint · TODO/FIXME > 90j

/test [chemin|module]

Analyse couverture + génère les 5 specs prioritaires (Vitest/Jest + Playwright/Cypress).

3 AGENTS

Inventaire (N unit, M e2e) · Couverture < 80% · Zones critiques sans test (routes API, stores, logique métier)

Design

/design

[sandbox|sonnet|gemini]

Mode design : audit DS + sandbox HTML interactif + prompts ciblés pour Sonnet ou Gemini Pro.

FLOW

- **Audit** (3 agents : tokens, hardcoding, primitives)
- **Sandbox HTML** interactif ouvert dans Chrome (palette, typo, spacing, composants live)
- **Question unique** : A (nettoyage) / B (évolution douce) / C (refonte) / D (3D avancé)
- **Prompt généré** : Sonnet (A/B/C) ou Gemini Pro (D, R3F + shaders)

PARTICULARITÉ

L'agent **ne touche pas au code**. Tu copies-colles le prompt dans une session dédiée.

/heist

<url>

Vol de design system. Scrape un site via Playwright headless, génère un thème drop-in.

CE QUE ÇA EXTRAIT

Top 24 couleurs · polices · typo scale · spacings observés · radius · shadows · motion · logo + 6 images · CSS custom properties exposées

FICHIERS GÉNÉRÉS

themes/<slug>/tokens.css · tokens.json · meta.md · sandbox.html · assets/

APPLY

<html data-theme="<slug">"> → tout le site bascule

/skin

<url>

Heist + intégration admin. Tes users finaux switchent de DS depuis l'UI, sans redeploy.

DIFFÉRENCE AVEC /HEIST

Lance `/heist` derrière, **puis** :

EN PLUS

- Détecte ton admin/settings, crée la page `/admin/themes`
- Composant `<ThemePicker />` avec grille de previews
- Table Supabase `user_theme_preferences` + RLS
- API `/api/user/theme` (POST + GET)
- `<ThemeHydrator />` qui charge le bon thème au boot
- Ajoute "Apparence" dans la nav admin

IDEMPOTENT

Relance autant de fois que tu veux pour ajouter d'autres thèmes

Compréhension

/brief (sans arg)

Briefing express. 30 secondes. 5 lignes pile. Le minimum syndical pour comprendre où tu es.

5 LIGNES EXACTES Projet + stack + branche · TL;DR du README · 3 derniers commits · WIP (propre ou N fichiers) · Prochain pas suggéré

/map [routes|stores|modules|api]

Cartographie complète du codebase. Compatible tous frameworks (Next, Svelte, Express, etc.).

4 VUES **Routes** (UI + API, méthode HTTP, auth) · **Stores** (Zustand/Redux/Pinia) · **Modules** (hubs critiques, circulaires) · **Surface API**

SORTIE Rapport HTML navigable + top 5 fichiers les plus couplés

/syscheck (sans arg)

Inspection système macOS. **Aucun kill auto.** Tout est proposé, tu confirmes par o.

5 PASSES Zombies · Doublons Node · Ports fantômes · Caches/orphelins · Logs lourds

WHITELIST AUTO `next dev` , `vite` , ports 3000-5173-8080-5432-54321, processus en cours

Orchestration & maintenance

/trio (sans arg)

Orchestrateur Opus. Tu pilotes 3 instances Sonnet 4.6 manuellement dans 3 fenêtres séparées.

WORKFLOW

- **Phase 1** : cadrage (but, contraintes, lectures)
- **Phase 2** : 1-3 prompts auto-suffisants à coller dans chaque Sonnet
- **Phase 3** : audit déclenché par `1` → VERROUILLÉ ou CORRECTIF

RÈGLE ABSOLUE Opus **ne touche jamais au code**. Décompose, audit, valide.

/tri

[--dry-run | --with-screenshots]

Coup de balai sur les .html/.md/.png orphelins. Toujours archive (**jamais** rm direct).

3 CATÉGORIES

PROTECTED (jamais touché : README, CLAUDE.md, refs entrantes) · **AUTO-DELETE** (mtime > 30j + 0 refs + pattern poubelle) · **REVIEW** (tu tranches)

ARCHIVE

`.tri-trash/YYYY-MM-DD-HHMM/` · purge manuelle quand tu es sûr

Stratégie

/horizon

[llm|db|auth|storage|...]

Scan ultra-profond : ton stack actuel vs l'état de l'art 2026. 10 agents spécialisés.

10 AGENTS

LLM (Claude 4.7, GPT-5, Gemini 3, AI SDK v5) · DB (Neon, Turso, Convex, Drizzle 1.0) · Auth (Better Auth, Clerk, Passkeys) · Storage (Tigris, UploadThing, Mux) · Real-time (Liveblocks, PartyKit) · Workflow (Trigger.dev v3, Temporal) · Observability (Helicone, Braintrust) · Frontend (Next 16, React 20, Panda CSS) · AI capabilities (Cartesia, Veo 3, Flux 1.1) · DevEx (Bun, Biome v2, tsgo)

RAPPORT HTML

Matrice **Actuel vs Alternative** · Heatmap pain points · Services à **virer** / **ajouter** · Roadmap visuelle (semaine 1 / mois 1 / trim)

SORTIE

Stack maturité /10 · Gain perf % · \$/mois économisable · deps retirables

Principes & conventions

Override projet > global

Si une commande existe à la fois en global (`~/ .claude/commands/`) et dans le projet (`.claude/commands/`), la version projet gagne automatiquement. Hearst OS a ses propres versions de `audit`, `qa`, `flow`, etc. avec des refs spécifiques (`spatial-safe`, `design system Hearst`).

Aucune question, mode autonomie

Toutes les commandes sont conçues pour ne **jamais** te poser de questions (sauf `/heist` et `/skin` qui demandent l'URL, et `/syscheck` qui confirme avant kill). Elles détectent le stack, les ports, les services et exécutent.

Read-only par défaut

Toutes les commandes d'audit/analyse sont read-only. Aucune modification de code sans demande explicite. `/syscheck` ne kill jamais sans `o.` `/tri archive` (`.tri-trash/`) au lieu de `rm`.

Rapports HTML standardisés

Toutes les commandes d'audit produisent un rapport HTML via `scripts/render-report.mjs` si présent, sinon JSON dans `/tmp/`. Le rapport est **auto-ouvert dans Chrome**.

Sous-agents typés et parallèles

Quand une commande spawn des sous-agents, elle utilise **un seul message avec N tool calls Agent** pour exécuter en parallèle.
`subagent_type`: `Explore` (read-only) est le défaut. Chaque agent retourne un livrable JSON structuré.

Severities P0/P1/P2 partout

P0 = bloquant prod, casse, faille. P1 = dégradation silencieuse, incohérence. P2 = dette mineure, dette stylistique. Format de finding unifié : `severity / path / line / rule / title / current / suggested / why`.

Catalogue services global

`~/ .claude/api-config/SERVICES.md` contient toutes tes clés API (Anthropic, Composio, Runway, etc.) — **réutilisé par /setup** sur chaque nouveau projet. Marqué dans `.gitignore`.

Globs anti-piège markdown

Tous les `grep/find` dans les commandes utilisent `*.tsx`, `*.ts`, `node_modules` sans escaping foireux. Les anciennes versions cassaient à cause du markdown qui mangeait les `*` comme italique.

Maintenance des commandes : éditer dans `~/ .claude/commands/<nom>.md` . Les commandes sont rechargées automatiquement à chaque session Claude Code.

Doc de conventions : `~/Dev/hearst-os/.claude/COMMANDS-CONVENTIONS.md` documente les conventions partagées.